

Semana 3: Estructuras de control y funciones

Nivel: Básico | **Prerrequisitos:** Semana 1 (secuencia/decisión/repetición, `if` / `while` / `for` básicos), Semana 2 (variables, tipos de datos, `input()` / `print()`, operadores) | **Versión:** v1.0 | **Fecha:** 2026-09-07

Objetivos de aprendizaje

1. Combinar condiciones con los operadores lógicos `and`, `or` y `not` para expresar decisiones compuestas, y anidar condicionales cuando una sola línea no alcanza.
2. Usar `range()` junto con `for` y controlar el flujo de un bucle con `break` y `continue`.
3. Definir funciones propias con `def`, parámetros (incluyendo valores por defecto) y `return`, para encapsular lógica reutilizable.
4. Distinguir el alcance (**scope**) local de una variable dentro de una función del alcance global del programa, y explicar por qué esa separación evita errores.
5. Refactorizar código repetido en funciones reutilizables, aplicando el principio DRY (**Don't Repeat Yourself**).

1. Decisiones compuestas: `and`, `or`, `not`

En la Semana 2 vimos condiciones simples: `edad >= 18`. La vida real casi nunca depende de una sola condición. Piensa en la regla de una alberca pública: **"puedes entrar si sabes nadar Y tienes más de 8 años, O si vienes acompañado de un adulto."** Esa frase combina dos condiciones con "Y" y una alternativa completa con "O" — exactamente lo que hacen `and` y `or` en Python.

```
sabe_nadar = True
edad = 6
acompanado_de_adulto = True

puede_entrar = (sabe_nadar and edad > 8) or acompanado_de_adulto
print(puede_entrar)    # True – porque va acompañado, aunque tenga 6 años
```

OPERADOR	SE CUMPLE CUANDO...	ANALOGÍA
<code>and</code>	ambas condiciones son verdaderas	Necesitas llave Y código para abrir una caja fuerte de doble seguridad.
<code>or</code>	al menos una condición es verdadera	Puedes pagar con tarjeta O efectivo: cualquiera de las dos sirve.
<code>not</code>	invierte el valor de una condición	Un interruptor que voltea <code>True</code> a <code>False</code> y viceversa.

```
print(not True)          # False
print(5 > 3 and 2 > 10)  # False → la segunda condición falla, así que "and" falla
print(5 > 3 or 2 > 10)   # True  → basta con que una sea verdadera
```

Cuidado con la precedencia: igual que en aritmética `*` se evalúa antes que `+`, en lógica `and` se evalúa antes que `or`. Cuando la regla combina ambos, usar paréntesis explícitos (como en el ejemplo de la alberca) no es opcional — es lo que evita que Python agrupe las condiciones de una forma distinta a la que tenías en mente.

Condicionales anidados

A veces una decisión depende de otra decisión ya tomada. Es la diferencia entre preguntar todo de una vez (`and`) y preguntar por pasos, como un formulario que solo muestra la segunda pregunta si respondiste "sí" a la primera:

```
tiene_cuenta = True
saldo = 150

if tiene_cuenta:
    if saldo >= 100:
        print("Retiro autorizado")
    else:
        print("Saldo insuficiente")
else:
    print("Primero debes abrir una cuenta")
```

Esto es equivalente a `if tiene_cuenta and saldo >= 100:` para el primer caso, pero anidar dos `if` dentro es más claro cuando cada nivel necesita **su propia** rama de "si no" — aquí "sin cuenta" y "con cuenta pero saldo insuficiente" son mensajes distintos, no uno solo.

2. `range()` y control fino del bucle

En la Semana 1, `for numero in numeros:` recorría una lista que ya existía. Muchas veces no hay una lista de antemano — solo se quiere repetir algo **N** veces. Para eso existe `range()`, que genera una secuencia de números sin necesidad de escribirla a mano:

```
for i in range(5):          # genera 0, 1, 2, 3, 4 (5 valores, empieza en 0)
    print(i)

for i in range(1, 6):      # genera 1, 2, 3, 4, 5 (desde 1 hasta 6, sin incluir el 6)
    print(i)

for i in range(0, 10, 2):  # genera 0, 2, 4, 6, 8 (de 2 en 2)
    print(i)
```

`break` y **`continue`** modifican el comportamiento por defecto del bucle:

- **`break`** corta el bucle por completo, de inmediato — como salir de una fila antes de tiempo porque ya encontraste lo que buscabas.
- **`continue`** salta solo la vuelta actual y sigue con la siguiente — como saltarte un plato que no te gusta en un bufet, sin dejar de recorrer el resto.

```
# break: buscar el primer número negativo en una lista
numeros = [4, 8, -3, 10, -7]
for n in numeros:
    if n < 0:
        print(f"Encontrado: {n}")
        break    # deja de revisar en cuanto encuentra el primero

# continue: sumar solo los números positivos, sin usar un if con else
suma = 0
for n in numeros:
    if n < 0:
        continue    # se salta esta vuelta, no suma el negativo
    suma = suma + n
print(f"Suma de positivos: {suma}")    # Suma de positivos: 22
```

Punto de confusión clásico: `break` y `continue` funcionan igual dentro de un `while` que dentro de un `for` — no son exclusivos de uno u otro. La diferencia está en qué tipo de bucle conviene según si conoces de antemano cuántas vueltas hará falta (ver la nota de la Semana 1 sobre `while` vs `for`).

3. Funciones: dejar de repetir código

Recuerda la calculadora de IMC de la Semana 2. Si quisieras calcular el IMC de tres personas distintas, copiarías y pegarías esas mismas líneas tres veces. Esa repetición es exactamente lo que una **función** evita — es la razón de ingeniería de software número uno para crear una: encapsular una tarea que se repite, en un solo lugar, con un nombre.

Analogía: una receta con nombre propio. En vez de reescribir "pelar, cortar y licuar la fruta" cada vez que alguien pide un jugo, el menú dice simplemente "Jugo de mango" — el nombre representa toda la secuencia de pasos. Llamar a una función es pedir "Jugo de mango"; la función es la receta completa detrás del nombre.

```
def calcular_imc(peso, estatura):  
    """Calcula el Índice de Masa Corporal a partir del peso (kg) y la estatura  
    (m)."""  
    imc = peso / (estatura ** 2)  
    return imc  
  
# Ahora "pedir el jugo" reemplaza copiar y pegar la fórmula:  
imc_ana = calcular_imc(68, 1.72)  
imc_luis = calcular_imc(90, 1.80)  
  
print(f"IMC de Ana: {imc_ana:.1f}")  
print(f"IMC de Luis: {imc_luis:.1f}")
```

Anatomía de una función:

- `def nombre_funcion(parametros):` — declara la función. `snake_case`, igual que las variables (Semana 2).
- Los **parámetros** (`peso`, `estatura`) son variables que solo existen mientras la función se ejecuta — son los "espacios en blanco" de la receta, que cada llamada rellena con sus propios valores.

- `return` entrega un resultado de vuelta a quien llamó la función, y termina la ejecución de la función en ese punto. Sin `return`, la función devuelve `None` (el equivalente en Python a "nada").
- Los valores que se pasan al llamar (`68`, `1.72`) se llaman **argumentos** — es la única diferencia de vocabulario: **parámetro** es el nombre dentro de la función, **argumento** es el valor real que se pasa al llamarla.

Parámetros con valores por defecto

Un parámetro puede tener un valor de respaldo, para cuando quien llama la función no quiere especificarlo cada vez:

```
def saludar(nombre, idioma="español"):
    if idioma == "español":
        print(f"Hola, {nombre}")
    elif idioma == "inglés":
        print(f"Hello, {nombre}")

saludar("Ana")           # usa el valor por defecto: "Hola, Ana"
saludar("John", "inglés") # sobrescribe el valor por defecto: "Hello, John"
```

4. Alcance (scope): qué variables existen dónde

Una variable creada **dentro** de una función es **local**: solo existe mientras esa función se ejecuta, y desaparece al terminar — es invisible desde afuera. Una variable creada **fuera** de cualquier función, a nivel del programa, es **global**: cualquier función puede leerla.

```
contador_global = 0    # variable global

def procesar_pedido():
    total_local = 100   # variable local: solo existe dentro de esta función
    print(total_local)

procesar_pedido()      # imprime 100
print(contador_global) # funciona: 0 – es global, visible en todo el programa
print(total_local)     # ERROR: NameError – total_local no existe fuera de la
                        # función
```

Analogía: el casillero del gimnasio (Semana 1), pero con reglas de acceso.

Una variable local es un casillero dentro de un vestidor privado: solo la persona que lo usó en ese momento puede verlo, y cuando esa persona se va, el casillero se vacía. Una variable global es un casillero en el pasillo principal: todos pueden verla y usarla. Esta separación no es una limitación molesta — es lo que permite que dos funciones distintas usen el mismo nombre de variable (`total_local` en una función, `total_local` en otra) sin que se pisen entre sí ni causen efectos colaterales inesperados en el resto del programa. En ingeniería de software esto se llama **encapsulamiento**: cada función es una caja cerrada que solo se comunica hacia afuera a través de sus parámetros y su `return`.

5. Por qué esto importa para IA/ML

Cada biblioteca de Machine Learning que usarás más adelante — `pandas`, `scikit-learn`, `PyTorch` — es, en el fondo, un enorme conjunto de funciones ya escritas por otras personas, cada una encapsulando una tarea específica (`entrenar()`, `predecir()`, `normalizar()`). Cuando dentro de unas semanas escribas `modelo.fit(datos)`, estarás llamando una función exactamente con la misma lógica que `calcular_imc(68, 1.72)` de hoy: le pasas argumentos, ella ejecuta una lógica interna que no necesitas reescribir, y te devuelve un resultado con `return`. Entender parámetros, `return` y **scope** hoy es lo que te permitirá leer y confiar en código ajeno de IA sin sentir que es magia. Los operadores lógicos también reaparecen tal cual: un sistema de recomendación real filtra candidatos con reglas como `precio < presupuesto and categoria == preferida`, la misma estructura del ejemplo de la alberca.

6. Ejemplo práctico: validador de contraseñas

Problema: escribir una función que revise si una contraseña cumple reglas mínimas de seguridad: al menos 8 caracteres, al menos un número, y no ser igual a `"12345678"` (una contraseña obviamente insegura).

Pseudocódigo:

```
FUNCION es_valida(contraseña)
  SI longitud de contraseña es menor a 8 ENTONCES
    devolver falso
  SI contraseña es igual a "12345678" ENTONCES
    devolver falso
  tiene_numero = falso
  PARA CADA caracter EN contraseña HACER
    SI caracter es un dígito ENTONCES
      tiene_numero = verdadero
    DETENER el bucle
  devolver tiene_numero
FIN FUNCION
```

Del pseudocódigo al código real (Python)

```
def es_valida(contrasena):
    """Devuelve True si la contraseña cumple las reglas mínimas de seguridad."""
    # DECISIÓN temprana: si falla algo obvio, salimos de inmediato con return.
    # Esto evita anidar todo el resto de la lógica dentro de un "else".
    if len(contrasena) < 8:
        return False
    if contrasena == "12345678":
        return False

    # REPETICIÓN + control fino: recorremos caracter por caracter buscando un
    # dígito.
    tiene_numero = False
    for caracter in contrasena:
        if caracter.isdigit():
            tiene_numero = True
            break # ya encontramos un número, no hace falta seguir revisando

    return tiene_numero

# Probamos la función con varios casos, sin repetir la lógica ni una sola vez:
candidatas = ["abc", "12345678", "contrasena", "contrasena9"]
for candidata in candidatas:
    resultado = "válida" if es_valida(candidata) else "inválida"
    print(f"{candidata}: {resultado}")
```

Resultado: `abc: inválida` (muy corta), `12345678: inválida` (está en la lista negra), `contrasena: inválida` (no tiene número), `contrasena9: válida`.

Punto de confusión clásico: un `return` dentro de un `for` (como pasaría si se hiciera `return True` en cuanto se encuentra un dígito) también termina la función entera, no solo el bucle — a diferencia de `break`, que solo termina el bucle y deja que el resto de la función siga ejecutándose. Aquí se usó `break` a propósito, porque después del bucle todavía falta el `return tiene_numero` final.

Segundo ejemplo: refactorizar el promedio de notas (Semana 2) en una función reutilizable

```
def evaluar_promedio(notas):  
    """Recibe una lista de notas y devuelve el promedio y si el estudiante  
    aprobó."""  
    promedio = sum(notas) / len(notas) # sum() sobre una lista: la ampliaremos en  
    la Semana 4  
    aprobado = promedio >= 6.0  
    return promedio, aprobado # una función puede devolver más de un  
    valor  
  
# Antes (Semana 2) había que copiar el bloque if/else por cada estudiante.  
# Ahora, un solo llamado por estudiante:  
for nombre, notas in [("Ana", [8.5, 7.0, 9.0]), ("Luis", [4.0, 5.5, 5.0])]:  
    promedio, aprobado = evaluar_promedio(notas)  
    estado = "Aprobado" if aprobado else "Reprobado"  
    print(f"{nombre}: promedio {promedio:.2f} – {estado}")
```

7. Notas de vigencia técnica (2026)

`def`, `return`, parámetros con valores por defecto, `range()`, `break` / `continue` y el modelo de **scope** local/global son parte del núcleo estable del lenguaje desde hace más de una década y siguen siendo la forma estándar de escribir Python en 2026 — nada de esto está en camino a quedar obsoleto. Un detalle de buena práctica moderna que sí vale la pena adoptar desde ahora: usar **docstrings** (`"""texto"""` justo debajo de `def`) para documentar qué hace cada función, como en los ejemplos de este capítulo — es la convención dominante en proyectos reales de IA/ML, y las herramientas de autocompletado e IA (incluidos asistentes como este) la usan para entender tu código.

8. Errores comunes de principiante

- Olvidar el `return` y esperar que la función "devuelva algo" por sí sola — sin `return` explícito, el resultado es `None`.
- Confundir `break` (corta el bucle entero) con `continue` (salta solo esta vuelta).

- Usar `or` cuando la intención lógica era `and` (o viceversa) — por ejemplo, escribir "mayor de edad `and` con acompañante" cuando la regla real permitía cualquiera de las dos condiciones por separado (`or`).
- Intentar usar fuera de una función una variable que fue creada dentro de ella (confundir alcance local con global).
- Escribir `range(5)` esperando que incluya el 5 — `range(n)` siempre se detiene un número antes de `n`.

9. Ejercicio propuesto

Escribe una función en Python llamada `clasificar_triangulo(lado1, lado2, lado3)` que reciba las tres longitudes de los lados de un triángulo y devuelva un texto:

```
"Equilátero" si los tres lados son iguales.  
"Isósceles" si exactamente dos lados son iguales.  
"Escaleno" si los tres lados son distintos.
```

Luego, usa un `for` para probar tu función con al menos cuatro conjuntos distintos de tres lados, e imprime el resultado de cada uno con una f-string.

Pista: vas a necesitar operadores de comparación, `and` / `or`, condicionales anidados o encadenados, y `return` — no imprimas dentro de la función, solo devuelve el texto y deja que quien la llama decida qué hacer con él.

10. Autoevaluación

1. ¿Cuál es la diferencia entre `and` y `or`? — `and` exige que ambas condiciones sean verdaderas; `or` solo exige que al menos una lo sea.
2. ¿Qué imprime `range(2, 9, 3)` si lo recorres con un `for`? — `2, 5, 8` (empieza en 2, suma de 3 en 3, se detiene antes de llegar o pasar 9).
3. ¿Cuál es la diferencia entre `break` y `continue`? — `break` termina el bucle por completo de inmediato; `continue` solo se salta la vuelta actual y sigue con la siguiente.
4. ¿Qué es un **parámetro** y qué es un **argumento**, y en qué se diferencian? — El parámetro es el nombre declarado dentro de la función (`def f(x):`); el argumento es el valor real que se pasa al llamarla (`f(5)`, aquí `5` es el argumento).
5. ¿Qué devuelve una función que no tiene ningún `return`? — `None`.

6. V/F: "Una variable creada dentro de una función puede usarse libremente fuera de ella." — Falso: es una variable local, solo existe mientras la función se ejecuta; usarla afuera produce `NameError`.
7. En el validador de contraseñas, ¿por qué se usó `break` y no `return True` dentro del `for`? — Porque `return` termina la función entera de inmediato, y después del bucle todavía faltaba ejecutar `return tiene_numero`; `break` solo termina el bucle y deja continuar el resto de la función.
8. Reescribe con `and` / `or`: "Puedes viajar si tienes pasaporte vigente y (visa o eres ciudadano del país destino)." — `tiene_pasaporte_vigente and (tiene_visa or es_ciudadano)`.
9. Explica con tus propias palabras por qué entender parámetros y `return` te prepara para usar bibliotecas de Machine Learning, sin usar jerga de programación. — Porque bibliotecas como scikit-learn o PyTorch son funciones ya escritas por otras personas: les pasas tus datos como argumentos y ellas devuelven un resultado con `return`, exactamente igual que `calcular_imc(68, 1.72)`, solo que la lógica interna es más compleja.

11. Verificación de dependencias

Este contenido depende de la Semana 1 (bloques de secuencia/decisión/repeticón, `if` / `while` / `for` en su forma básica) y de la Semana 2 (variables, tipos de datos, operadores de comparación, `input()` / `print()`, f-strings): los operadores lógicos extienden directamente los condicionales de la Semana 2, `range()` / `break` / `continue` refinan la repetición ya introducida en la Semana 1, y las funciones se construyen reutilizando ejemplos ya resueltos en ambas semanas (la calculadora de IMC, el promedio de notas) sin introducir ningún concepto matemático o de programación no visto antes. Es prerrequisito directo de la Semana 4 (estructuras de datos fundamentales: listas, diccionarios, tuplas, conjuntos), que asume que ya se sabe definir y llamar funciones, y usar bucles con control de flujo (`break` / `continue`) para recorrer colecciones.